[CRITICAL] Plan: HymeKo-Nagare → HSMM Compiler

Future-session work; FPGA prerequisite

Drafted: 2026-05-11. Status: **frozen**, awaiting FPGA HSMM kernel.

Status banner

This is a **CRITICAL** plan flagged for a **future session**. It is **not** actionable in the current session because:

1. The **HSMM FPGA implementation is a prerequisite**. HSMM (Hypergraph Storage Modification Machine, Hajdu & Csapó 2026 Zenodo manuscript) currently has a Zynq UltraScale+ kernel referenced in `docs/plans/plans_20260429/hymeko_monitor_plan.md` but the bitstream / HDL artefacts are not in this repo (verified via `grep`). The compiler target must exist before the source-language compiler is meaningful.
2. The Nagare ops landed today are CPU-only; GPU backend (Stage 3 of `docs/plans/2026-05-11-hymeko-nag` has not yet been built. Going straight to FPGA would skip a natural staging point.
3. Empirical validation requires a Nagare → HSMM end-to-end pipeline that we can compare against PyTorch-on-CPU on real signed-graph workloads — requires the FPGA target booted and the Nagare ops mature.

Re-open this plan when: the HSMM FPGA bitstream is in repo (`hymeko_monitor/hsmm_bridge.rs` mentions it; needs to materialise), OR the Hajdu-Csapó companion paper publishes a public HSMM reference implementation, whichever comes first.

1 Goal

Build a compiler ϕ : Nagare-program → HSMM-program such that every Nagare training-loop function (forward + closed-form backward + Adam step + loss) compiles to a fixed HSMM-program (sequence of NEWE / ATT / arithmetic primitives) executable on the Zynq UltraScale+ HSMM kernel.

This is the systems counterpart to the algorithmic paper `docs/plans/2026-05-11-hymeko-nagare-flow/plan` it demonstrates that Nagare is the natural ML programming language for HSMM (Schönhage-class storage-modification machine, hypergraph variant), via a structural embedding rather than a trace-based lowering.

2 Why this matters

For PyTorch / JAX / Burn: reverse-mode autograd needs to traverse the recorded computation graph during backward. Compiling that to HSMM would require lowering the graph traversal itself to HSMM primitives — effectively building an HSMM-resident autograd interpreter. That is feasible but bypasses the whole point of HSMM (compiled, fixed-schedule execution).

For Nagare: the closed-form Clifford backward (Theorem 1 of `math.tex` in the Nagare plan dir) is already expressed as a fixed sequence of READ / FMA / ATT-LOCAL operations over the hyperedge stream. There is no graph traversal at backward time. The compilation to HSMM is a

structural embedding (operator → template), not a *trace-based* one. This is the property that makes the Nagare → HSMM pairing structurally clean.

3 Affected files (when reopened)

```
hymeko_nagare/
        src/
            compiler/
                mod.rs              pub use of the compiler
                ir.rs               Nagare op IR (op-graph in flat Vec form)
                hsmm_target.rs      HSMM program emit (NEWE / ATT / FMA)
                schedule.rs         operator-level scheduling for FPGA pipelin
                verify.rs           structural-equivalence checker (Nagare op
                                    vs emitted HSMM program)
            ...                     (existing ops crate)
hymeko_monitor/
        src/
            hsmm_bridge.rs          existing scaffold; will host the runtime
                                    bridge between Nagare-compiled HSMM
                                    programs and live FPGA events.
docs/plans/CRITICAL-nagare-to-hsmm-compiler/
        plan.tex (this doc)
        plan.pdf
        plan.tikz                   Nagare-IR -> HSMM-program structural diagram
        plan.mmd                    compiler stages flowchart
        README.md                   prerequisites checklist + reopen criteria
```

4 Prerequisites checklist

Before reopening this plan, ALL must be true:

1. HSMM Zynq bitstream OR public reference implementation available in repo, with documented NEWE / ATT primitive surface (event format, memory layout, dispatch protocol).
2. Nagare Stage 3 (GPU backend via wgpu OR cudarc) shipped, so we have a CPU → GPU step *before* CPU → FPGA; that gives us 2 architectural-portability validation points before the FPGA attempt.
3. Catmull-Rom spline forward + closed-form backward ported to pure Rust (Nagare Stage 4), so the spline-aggregator path is also compilable to HSMM. Without this, only the FIR path works on HSMM, and the comparison vs the full Slashdot SOTA recipe is not meaningful.
4. Slashdot 5-seed paired AUC reproduced on Nagare-CPU pipeline at parity (≤ 0.005 AUC drift) vs the existing PyTorch + Triton kernel pipeline. This validates that Nagare's pure-Rust ops have no accuracy regression before attempting FPGA deployment.

5 Compilation stages (when reopened)

1. **Parse Nagare op chain.** Take a Rust function of type `fn(&CyclePool, &Features) -> &Logits` composed of registered Nagare ops (no autograd traces, no closures with captured state). Produce a flat Nagare-IR.

2. **Verify closure under op registry.** Every node in the IR must be a registered Nagare operator with declared forward + closed-form backward kernels. Reject if any op is autograd-traced (which would require an interpreter on FPGA).

3. **Lower to HSMM-IR.** For each Nagare op, instantiate the corresponding HSMM-template (per the structural embedding ϕ defined in `math.tex` Section 7). Templates are parameterised by (cycle batch size, feature dim, filter length, optional Highway/attention sub-templates if Stage 4 has landed).

4. **Schedule for FPGA pipeline.** Assign operators to FPGA pipeline stages; pipeline-balance via fanout / fanin blockers; check resource budget (DSP slices, BRAM tiles, LUT-FF pairs) against the Zynq UltraScale+ chip-specific budget.

5. **Emit Verilog / SystemVerilog** via the existing `hymeko_monitor` LIR → SystemVerilog path (see plan `docs/plans/plans_20260429/hymeko_monitor_plan.md`). The Nagare compiler reuses this emission step — it just emits a different LIR program shape.

6. **Bitstream synthesis + deploy.** Standard Xilinx Vivado pipeline. Out of scope for the Nagare compiler crate, in scope for the integration tests.

7. **Empirical validation.** Run Slashdot or Epinions forward + backward pass on the FPGA-resident compiled Nagare program; report throughput (cycles/s, edges/s) and compare against PyTorch CPU baseline + Nagare CPU baseline. Expected range: 10–100× over PyTorch CPU at iso-FLOPS, with much lower energy/op (FPGA-typical 10–50× better than CPU at signal-processing-class workloads, which the FIR + scatter primitives are).

6 Test strategy

Compiler unit tests (pure-CPU, no FPGA):

- Each Nagare-op → HSMM-template mapping has a unit test that emits the HSMM program and verifies it against a hand-written reference HSMM program for that op.
- Round-trip: Nagare program → HSMM-IR → executed in a Rust HSMM-IR interpreter → result matches Nagare CPU forward to $\leq 10^{-6}$.
- Backward parity: emitted HSMM program for backward matches Nagare CPU closed-form backward to $\leq 10^{-6}$.

FPGA integration tests (requires bitstream on Zynq):

- Slashdot k=3 cycle pool, forward only, output parity $\leq 10^{-5}$ vs Nagare CPU. (Float-precision drift from FPGA fixed-point arithmetic, if used, is the limiting factor.)
- Full training loop (forward + bwd + Adam step) for 10 epochs, AUC parity ≤ 0.01 vs Nagare CPU.
- Throughput measurement: edges processed per second; target $\geq 10\times$ PyTorch CPU at iso-FLOPS.

7 Performance budget

- Compile time: ≤ 30 s for a typical Nagare training loop (parsing + lowering + verilog emission). FPGA synthesis via Vivado is hours; that's not the compiler's wall.
- FPGA latency: ≤ 1 ms per forward pass on Bitcoin OTC scale (30K cycles, k=3, d=32); proportionally for larger.
- FPGA energy: ≤ 1 W per training step (compared to 50–100 W for the equivalent on a 7nm CPU). The energy-cost argument is the headline for the deployment / edge-inference pitch.

8 Rollback path

If FPGA deployment proves infeasible for any reason (Vivado bug, resource budget overrun, fixed-point precision unacceptable for AUC parity), Nagare's CPU + wgpu/cudarc backends remain fully functional. The compiler crate is a separate module; deleting it does not regress the Nagare CPU+GPU path. Worst case: the FPGA compiler is shelved indefinitely, Nagare ships as a CPU+GPU framework, and the Hajdu-Csapó HSMM paper stands on its own (without the Nagare integration as proof-of-feasibility).

9 Risk anticipation

- **Fixed-point precision.** FPGAs typically use fixed-point arithmetic; f32 emulation eats DSP slices. AUC parity at fixed-point Q16.16 vs f32 needs early validation (Stage 6 in the compilation pipeline).
- **HSMM primitive surface drift.** The NEWE/ATT primitive set in the Hajdu-Csapó manuscript may evolve before publication. The compiler should target an explicitly versioned HSMM-IR (`hsmm_v1.0`) so the source-language remains stable when the target evolves.
- **Scope creep into attention.** HSiKAN's quaternion-attention edge-cr Highway path has a more complex closed-form backward; whether to support attention in the Nagare compiler depends on whether attention proves necessary for SOTA-on-FPGA. Default: ship the FIR-only path first.
- **HSMM Zynq performance ceiling.** Zynq UltraScale+ has $\sim$ 2000 DSP slices; large cycle pools (Epinions 530K cycles) may exceed throughput at d=32. Mitigated by streaming the cycle pool in chunks of 10–50K cycles (the existing `HSIKAN_CYCLE_BATCH=2000` pattern), which fits the DSP budget comfortably.

10 Venue strategy

This work, when shipped, completes a three-paper arc:

1. **Hajdu & Csapó 2026 (Zenodo, in review):** HSMM as abstract computational model. Pure theory.
2. **Nagare paper (planned 2026, MLSys 2027):** Cycle-additive operators with closed-form Clifford backward; structural correspondence to HSMM via embedding ϕ. Theory + CPU implementation.
3. **This plan (Nagare-to-HSMM compiler, planned 2027):** Empirical realisation: compiled Nagare programs running on Zynq UltraScale+ HSMM kernel, with throughput + energy measurements vs PyTorch CPU + Nagare GPU baselines. Target venues: FPGA 2028 / FCCM 2028 / DAC 2028 / FCCM 2028; or ASPLOS 2028 if the framing leans architecture rather than EDA.

The three papers together establish HSMM as a viable parallel-ML substrate with a working software stack and empirical advantage, a much stronger claim than HSMM theory alone.

11 Citation arc

- Schönhage 1980 (original Storage Modification Machine)
- Knuth-Tarjan pointer machines (1979)
- Kolmogorov machines (1953)

- Hajdu & Csapó 2026 (HSMM = hypergraph SMM, Zenodo)
- HymeKo-Nagare 2026 (`docs/plans/2026-05-11-hymeko-nagare-flow/`)
- This plan (Nagare → HSMM compiler, 2027 work)